

BIG DATA ANALYTICS

Mini Project 2 — Spark Case Study

NYC Yellow Taxi Trip Data Analysis

Submitted by:

Mariam Mohamed Goda | 202201223
Yasmin Osama | 202201683

April 2026

1. Project Overview

1.1 Objective

This report presents the results of Mini Project 2 for the Big Data Analytics course. The goal was to analyze large-scale taxi trip data using Apache Spark — implementing each query in three different APIs (RDD, DataFrame, and Spark SQL) — and to compare their execution plans and performance characteristics.

1.2 Research Question

How do fare amounts, trip distances, passenger volumes, and payment patterns vary across time of day, day of week, and geographic zones in NYC — and what Spark optimizations best support large-scale analysis?

1.3 Dataset Description

Source: Kaggle — [elemento/nyc-yellow-taxi-trip-data](#)
Files used: yellow_tripdata_2015-01.csv (January 2015)
Total Records: ~11.4 million taxi trips
Columns: 19 attributes including timestamps, coordinates, fare components, and payment info

Attribute	Details
Size	~11.4 million records (full dataset)
Columns	19 columns — timestamps, floats, integers, strings
Format	CSV (converted to Parquet in Query 10)
Date Range	January 2015
Key Columns	VendorID, pickup/dropoff datetime, fare_amount, trip_distance, passenger_count, payment_type, tip_amount, total_amount

1.4 Implementation Approach

Each of the 10 queries was implemented in three ways:

- RDD API: Low-level, functional-style transformations with Python lambdas
- DataFrame API: High-level structured API with Catalyst optimizer
- Spark SQL: Declarative SQL queries executed through the Catalyst optimizer

A 20% random sample (fraction=0.2, seed=42) was used for all queries to ensure stable execution on a local single-node VM with limited resources, while still processing millions of records.

Query 1: Filtering with Complex Conditions

Type: Filtering | **Problem:** Trips where fare > \$20 AND passengers > 1

Results

- RDD: 79,679 trips matched | Time: 11.88s
- DataFrame: 79,679 trips matched | Time: 1.45s
- Spark SQL: 79,679 trips matched | Time: 2.99s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	11.88s	1.45s	2.99s
Result Count	79,679	79,679	79,679
Predicate Pushdown	No	Yes	Yes
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD performs a full Python scan with no optimization. DataFrame/SQL use Catalyst predicate pushdown to filter rows before scanning the full dataset. DataFrame is 8.2x faster than RDD.

Analytical Finding

Premium trips (fare > \$20, passengers > 1) account for roughly 1.8% of all sampled trips, suggesting most NYC taxi rides are short single-passenger trips. The highest fares reached \$780, indicating airport or long-distance trips.

Query 2: Aggregations per Payment Type

Type: Aggregation (SUM, AVG, COUNT, MAX, MIN) | **Problem:** Fare statistics grouped by payment type

Results

- RDD: 4 payment types computed | Time: 18.15s
- DataFrame: 4 payment types | Time: 0.25s
- Spark SQL: 4 payment types | Time: 0.24s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	18.15s	0.25s	0.24s
Aggregation	Manual Tuples	Built-in	Built-in
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD requires manual accumulator tuples to compute multiple metrics simultaneously. DataFrame/SQL use HashAggregate with Tungsten binary format — 72x faster than RDD. SQL is marginally faster due to query plan caching.

Analytical Finding

Credit card payments (type 1) dominate with 1,381,058 trips and avg fare \$12.49. Cash payments (type 2) show 844,051 trips at \$10.93 avg. No-charge and dispute types are rare. This suggests a strong preference for card payments in NYC taxis.

Query 3: Revenue by Hour of Day and VendorID

Type: Grouping by Multiple Attributes | **Problem:** Total revenue grouped by pickup hour and vendor

Results

- RDD: Top 20 hour+vendor pairs computed | Time: 129.17s
- DataFrame: Full result | Time: 0.33s
- Spark SQL: Full result | Time: 0.69s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	129.17s	0.33s	0.69s
Grouping Keys	(hour, vendor)	(hour, vendor)	(hour, vendor)
Optimizer	None	Catalyst	Catalyst

Key Insight

Multi-key GROUP BY requires composite key tuples in RDD — extremely slow at 129s. DataFrame/SQL handle multi-column grouping natively with HashAggregate. DataFrame is 391x faster than RDD.

Analytical Finding

Evening hours (18:00-22:00) with Vendor 2 (VeriFone) generate the highest revenue, peaking at \$851,601 at hour 18. This reveals that dinner-time trips are the most lucrative window for drivers and fleet operators.

Query 4: Top 10 Most Expensive Trips

Type: Sorting and Ranking | **Problem:** Top 10 trips ordered by fare amount descending

Results

- RDD: Top 10 found (max fare \$999.99) | Time: 35.95s
- DataFrame: Top 10 found | Time: 0.20s
- Spark SQL: Top 10 found | Time: 0.18s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	35.95s	0.20s	0.18s
Sort Strategy	Full Sort	Min-Heap (TakeOrdered)	Min-Heap (TakeOrdered)
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD's sortBy() sorts the entire dataset before taking 10 rows — $O(N \log N)$ cost. Catalyst uses TakeOrderedAndProject, a min-heap of size 10 that avoids full shuffling. SQL is fastest at 0.18s — 200x faster than RDD.

Analytical Finding

The most expensive trip was \$999.99 with 0 distance recorded, suggesting a flat-rate or disputed fare. Multiple \$900 fares appear with single passengers and zero distance, which may indicate pre-booked flat fares or data quality issues worth investigating.

Query 5: 7-Day Moving Average of Fare

Type: Window Function (Moving Average) | **Problem:** 7-day sliding window average of daily fare amounts

Results

- RDD: 31 days computed | Time: 22.32s
- DataFrame: Full result | Time: 0.34s
- Spark SQL: Full result | Time: 0.46s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	22.32s	0.34s	0.46s
Window Function	Manual (driver)	Built-in	Built-in
Distributed	Partial	Yes	Yes
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD computes daily averages on the cluster but must collect to the driver to apply the 7-day sliding window — breaking distributed processing. DataFrame/SQL use the Tungsten Window operator natively, keeping computation fully distributed.

Analytical Finding

Daily fares in January 2015 averaged around \$12.00-\$12.76 with minimal weekly variation. The 7-day moving average smooths daily fluctuations, revealing a slight downward trend mid-month (\$11.88 by Jan 10), possibly due to post-holiday travel decline.

Query 6: Trips Above Average Fare per Rate Code

Type: Nested Subquery | **Problem:** Find trips whose fare exceeds the average fare of their rate code

Results

- RDD: 3,942,347 above-avg trips | Time: 239.16s
- DataFrame: 3,942,347 trips | Time: 36.09s
- Spark SQL: 3,942,347 trips | Time: 29.83s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	239.16s	36.09s	29.83s
Result Count	3,942,347	3,942,347	3,942,347
Join Strategy	Broadcast (manual)	Auto (Catalyst)	Auto (Catalyst)
Optimizer	None	Catalyst	Catalyst

Key Insight

The subquery computes per-rate averages and joins back to the main table. RDD requires two passes plus a broadcast — 239s. Catalyst automatically detects the small aggregation result and applies BroadcastHashJoin, making SQL 8x faster than RDD.

Analytical Finding

About 34% of all trips (3.9M out of ~11.4M) have fares above their rate code's average, meaning a significant portion of riders pay above-average fares. This is useful for detecting premium demand zones or time periods.

Query 7: Broadcast Join vs Sort-Merge Join

Type: Join Optimization | **Problem:** Enrich trips with borough names — compare join strategies

Results

- RDD: Manual dict broadcast | Time: 88.03s
- DataFrame Broadcast Join: Time: 11.18s
- Sort-Merge Join: Time: 15.55s | SQL BROADCAST hint: similar to BJ

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	88.03s	11.18s (BJ) / 15.55s (SMJ)	~11s
Shuffle Required	No	No (BJ) / Yes (SMJ)	No
Sort Step	No	No (BJ) / Yes (SMJ)	No
Best For	Small lookup	Small table (<10MB)	Large tables

Key Insight

Broadcast Join sends the small borough lookup (8 rows) to all executors — no shuffle needed. Sort-Merge Join sorts and shuffles both sides — 39% slower. With tiny lookup tables, Broadcast Join is always preferred.

Analytical Finding

The borough lookup confirms that the majority of pickup coordinates map to Manhattan and Brooklyn zones, consistent with real NYC taxi demand patterns. Broadcast Join is 1.39x faster than Sort-Merge for this small-table scenario.

Query 8: Cumulative Daily Revenue

Type: Window Function (Cumulative Sum) | **Problem:** Running total of daily revenue across the dataset

Results

- RDD: Cumulative computed on driver | Time: 70.61s
- DataFrame: Distributed cumulative sum | Time: 0.28s
- Spark SQL: Native window cumulative sum | Time: 0.92s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	70.61s	0.28s	0.92s
Window Function	Manual (driver)	Built-in	Built-in
Distributed	No	Yes	Yes
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD collects all daily revenue to the driver node before computing the cumulative sum sequentially — a bottleneck that cannot scale. DataFrame uses ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW natively, 252x faster.

Analytical Finding

Daily revenue in January 2015 reached cumulative totals in the tens of millions by month-end, reflecting the massive scale of NYC taxi operations. DataFrame's window function is by far the best tool for running total calculations.

Query 9: Busiest Pickup Zones by Day of Week

Type: Grouping + Window Ranking | **Problem:** Top 5 busiest pickup areas for each day of the week

Results

- RDD: Manual sort + Python top-N | Time: 102.93s
- DataFrame: RANK() window function | Time: 0.49s
- Spark SQL: RANK() OVER PARTITION BY | Time: 0.37s

Performance Comparison

Metric	RDD	DataFrame	Spark SQL
Execution Time	102.93s	0.49s	0.37s
Top-N Strategy	Manual collect+sort	RANK()	RANK()
Grouping Keys	(day, area)	(day, area)	(day, area)
Optimizer	None	Catalyst	Catalyst

Key Insight

RDD must collect all (day, zone) counts to the driver for Python-side sorting — a massive bottleneck at 103s. Catalyst pushes RANK() window into a single distributed pass. SQL is the fastest at 0.37s, 278x faster than RDD.

Analytical Finding

Zone 237 (Upper East Side) and Zone 161 (Midtown) consistently rank among the top pickup areas regardless of day of week. Weekend patterns show higher demand in entertainment districts, while weekdays peak near office zones.

Query 10: CSV vs Parquet Performance Benchmark

Type: File Format Optimization + Caching + Partition Pruning

Benchmark Results

Strategy	Time	vs CSV Baseline
RDD (CSV)	101.86s	baseline
DataFrame – CSV (no cache)	9.59s	baseline
DataFrame – Parquet (no cache)	1.21s	7.9x faster
DataFrame – Parquet + Cache	1.04s	9.2x faster
DataFrame – Partitioned Parquet (pruned)	0.83s	11.5x faster

Key Insights

- Parquet is 7.9x faster than CSV due to columnar storage and compression — only needed columns are read from disk.
- Caching eliminates disk I/O entirely for repeated queries — Parquet + Cache is 9.2x faster than CSV.
- Partition pruning (reading only trip_month=1) skips all other partitions — the fastest strategy at 11.5x.
- RDD on CSV is the slowest at 101.86s — no columnar optimization, no Catalyst, and full Python deserialization.
- Parquet stores column-level min/max statistics enabling predicate pushdown at the file level.

Analytical Finding

For production Spark workloads, Parquet with partition pruning is the optimal storage strategy. The 11.5x speedup over CSV means that a query taking 2 hours on raw CSV would complete in about 10 minutes with partitioned Parquet — a transformative performance improvement for large-scale analytics.

Performance Summary — All 10 Queries

The table below summarizes execution times (in seconds) across all queries and APIs. The asterisk (*) marks the fastest API per query.

Query	RDD (s)	DataFrame (s)	Spark SQL (s)
Q1 – Filtering (fare>20 & pass>1)	11.88	1.45 *	2.99
Q2 – Aggregations per payment type	18.15	0.25 *	0.24 *
Q3 – Revenue by hour + VendorID	129.17	0.33 *	0.69
Q4 – Top 10 most expensive trips	35.95	0.20	0.18 *
Q5 – 7-day moving avg fare	22.32	0.34 *	0.46
Q6 – Above-avg fare per RateCode	239.16	36.09	29.83 *
Q7 – Broadcast join (borough lookup)	88.03	11.18 *	~11
Q8 – Cumulative daily revenue	70.61	0.28 *	0.92
Q9 – Busiest zones by day of week	102.93	0.49	0.37 *
Q10 – Parquet vs CSV (best strategy)	101.86	0.83 *	9.59 (CSV)

* = fastest API for that query

Optimization Analysis

Catalyst Optimizer

The Catalyst optimizer is the backbone of Spark's structured APIs. It transforms logical plans through four stages:

- Analysis: Resolves column names, data types, and table references against the catalog
- Logical Optimization: Applies rules such as predicate pushdown, constant folding, and projection pruning
- Physical Planning: Selects the best physical operator (e.g., BroadcastHashJoin vs SortMergeJoin) based on statistics
- Code Generation: Uses Tungsten's whole-stage code generation to compile query plans to optimized JVM bytecode

RDD bypasses all of these stages. Every RDD transformation is a black box to Spark — it cannot reorder, combine, or optimize Python lambda functions.

Join Optimization: Broadcast vs Sort-Merge

Query 7 demonstrated the critical difference between join strategies:

Property	Broadcast Join	Sort-Merge Join
Shuffle	None — small table sent to all executors	Required — both sides shuffled by join key
Sort Step	Not required	Both sides sorted before merge
Best For	One table < 10MB (configurable threshold)	Two large tables of similar size
Time (Q7)	11.18s	15.55s (39% slower)

File Format Impact: CSV vs Parquet

Query 10 quantified the massive impact of file format choice:

- CSV is row-based — Spark must deserialize every column even if only one is needed
- Parquet is columnar — only the requested columns are read from disk
- Parquet includes embedded schema — no inferSchema overhead
- Parquet files include min/max statistics per row group, enabling predicate pushdown at the file level
- Parquet with partitioning (by trip_month) allows Spark to skip entire directories — 11.5x faster than CSV

Caching Strategy

When the same DataFrame is queried multiple times, caching stores the result in memory after the first computation. Subsequent queries skip disk I/O entirely. This was demonstrated in Query 10 where Parquet + Cache (1.04s) outperformed uncached Parquet (1.21s).

RDD vs DataFrame vs SQL — Final Verdict

Criterion	RDD	DataFrame	Spark SQL
Optimizer	None	Catalyst	Catalyst
Performance	Slowest	Fast	Fastest (most queries)
Ease of Use	Hard	Easy	Easiest
Window Functions	Manual only	Native	Native
Join Optimization	Manual broadcast	Auto (Catalyst)	Auto + hints
Type Safety	None	Compile-time	Runtime

Conclusions and Lessons Learned

Business Insights from the Dataset

- Credit card is the dominant payment method (62% of trips), confirming the shift from cash to digital payments in NYC transportation.
- Evening hours (18:00-21:00) generate the highest revenue per vendor, making this the most valuable time window for taxi operators.
- About 34% of trips exceed their rate code's average fare, suggesting significant variability driven by traffic, distance, and time of day.
- The top 10 most expensive fares (\$672-\$999) show zero trip distance, likely representing pre-booked flat fares, airport transfers, or data quality issues.
- January 2015 daily average fares remained stable around \$11.88-\$12.76, indicating consistent pricing with minor weekly fluctuations.

Lessons Learned About Spark Optimization

- RDD is never the right choice for structured data. Even simple aggregations are 18-391x slower than DataFrame/SQL due to lack of Catalyst optimization.
- Parquet format delivers 7.9x read speedup over CSV with zero code changes — simply changing the file format is the highest-ROI optimization available.
- Partition pruning can reduce query time by 11.5x by skipping entire file partitions, making it essential for time-series data.
- Broadcast Join should always be used when one side of a join is small (< 10MB default threshold), saving significant shuffle cost.
- Caching is essential for iterative workloads. If the same DataFrame is used multiple times, `cache()` pays for itself immediately.
- Window functions (RANK, moving average, cumulative sum) should never be implemented manually in RDD — Spark's native window operators are orders of magnitude faster.
- The Catalyst optimizer's predicate pushdown, join reordering, and code generation make DataFrame/SQL the clear winners for all analytical workloads.

Recommendation

For production big data workloads: store data in partitioned Parquet format, use DataFrame or Spark SQL APIs exclusively, leverage broadcast joins for dimension tables, and cache frequently reused DataFrames. This combination delivers the best performance with the least code complexity.